

Python (15)

Q1. What is Python?

Python is a high-level interpreted language known for readability and fast development. It is widely used in web development, automation, and data science. It has a large ecosystem of libraries.

Q2. Dynamic typing?

Python is dynamically typed, so variables don't need explicit type declaration. The type is decided at runtime. This makes development faster and flexible.

Q3. List vs Tuple?

List is mutable and can be modified. Tuple is immutable and cannot be changed. Tuples are faster and used for fixed data.

Q4. Dictionary?

Dictionary stores data as key-value pairs. Keys are unique and allow fast lookup. It is widely used for structured data.

Q5. Function?

Function is a reusable block of code. It improves modularity and reduces repetition. It can take inputs and return outputs.

Q6. Lambda?

Lambda is a small anonymous function written in one line. It is used for short operations and improves code conciseness.

Q7. List comprehension?

List comprehension is a compact way to create lists. It improves readability and replaces loops in many cases.

Q8. Slicing?

Slicing is used to extract parts of a sequence. It uses start:end syntax and is useful for data manipulation.

Q9. Mutable vs immutable?

Mutable objects can change, immutable cannot. Lists are mutable, tuples and strings are immutable.

Q10. Set?

Set is an unordered collection of unique elements. It removes duplicates automatically.

Q11. args?

*args allows variable number of positional arguments. It is useful for flexible functions.

Q12. kwargs?

**kwargs allows variable keyword arguments. It stores them as dictionary.

Q13. Recursion?

Recursion is when a function calls itself. It solves problems in smaller parts.

Q14. Module?

Module is a file containing Python code. It helps organize code and reuse it.

Q15. pip?

pip is Python package manager. It installs and manages libraries easily.

Django (15)

Q1. What is Django

Django is a high-level Python web framework used to build secure and scalable web applications quickly. It follows the MVT architecture and comes with built-in features like authentication and an admin panel. It helps developers focus on logic instead of boilerplate code.

Q2. What is MVT architecture?

MVT stands for Model, View, Template. The Model handles database structure, the View processes

logic, and the Template handles UI. This separation makes code organized and maintainable. It is similar to MVC but structured differently.

Q3. What is Model?

A Model defines the structure of the database in Django. Each model is a Python class where attributes represent table fields. Django automatically converts it into a database table. It also provides ORM methods for querying data.

Q4. What is View?

A View handles incoming requests and returns responses. It contains business logic and interacts with models. Views can return HTML templates or JSON data. They are essential for controlling application flow.

Q5. What is Template?

Templates are HTML files used to display data. Django uses template language to insert dynamic content. It allows loops, conditions, and variables inside HTML. This keeps logic separate from UI.

Q6. What is ORM?

ORM allows developers to interact with the database using Python code instead of SQL. It simplifies queries and improves readability. It also helps prevent SQL injection. Django ORM is powerful and widely used.

Q7. What are migrations?

Migrations are used to apply changes in models to the database schema. When models are updated, migrations help sync changes with the database. Commands like makemigrations and migrate are used. It ensures version control of DB structure.

Q8. What is Django Admin?

Django Admin is a built-in interface to manage database records. It allows adding, updating, and deleting data easily. Developers just need to register models. It saves time by avoiding custom admin panels.

Q9. What is URL routing?

URL routing maps URLs to specific views. It helps Django decide which function should handle a request. URLs are defined in `urls.py` file. Clean URL structure improves application design.

Q10. GET vs POST?

GET is used to retrieve data and is visible in the URL. POST is used to send data securely and is not visible in the URL. POST is commonly used in forms. GET is faster but less secure.

Q11. What is QuerySet?

A QuerySet is a collection of database queries. It allows filtering, retrieving, and manipulating data. QuerySets are lazy, meaning they execute only when needed. They make DB operations easy.

Q12. Static vs Media files?

Static files include CSS, JS, and images used in UI. Media files are user-uploaded content like profile pictures. Django handles both differently using separate configurations. Proper handling is important in deployment.

Q13. What is Middleware?

Middleware is a layer that processes requests before reaching views and responses before returning to users. It is used for authentication, logging, etc. It runs globally across the project. It helps manage cross-cutting concerns.

Q14. What is DRF?

Django REST Framework is used to build APIs. It allows sending data in JSON format to frontend apps like React. It provides serializers, authentication, and easy API creation. It is widely used in modern web apps.

Q15. What is Serializer?

A serializer converts complex data like models into JSON format and vice versa. It is used in APIs to send and receive data. It also validates input data. It plays a key role in DRF.

React (15)

Q1. What is React?

React is a JavaScript library used to build user interfaces, especially single-page applications. It uses components to create reusable UI elements. React improves performance using virtual DOM. It is widely used with backend APIs.

Q2. What is JSX?

JSX is a syntax extension that allows writing HTML-like code inside JavaScript. It makes UI code easier to read and write. JSX is compiled into JavaScript by Babel. It is a core concept in React.

Q3. What is component?

Components are reusable building blocks of a React app. Each component represents a part of the UI. They can be functional or class-based. Reusability improves development speed.

Q4. What is state?

State is used to store dynamic data inside a component. When state changes, the component re-renders automatically. It is managed using hooks like `useState`. It is essential for interactive UI.

Q5. What are props?

Props are used to pass data from parent to child components. They are read-only and cannot be modified by child components. Props help in communication between components. They improve modularity.

Q6. State vs Props?

State is managed inside a component and can change. Props are passed from parent and are immutable. State is used for dynamic data, while props are for communication. Both are essential in React.

Q7. `useState`?

`useState` is a hook used to manage state in functional components. It returns a state variable and a function to update it. It makes functional components powerful. It replaces class-based state handling.

Q8. `useEffect`?

`useEffect` is used to perform side effects like API calls or DOM updates. It runs after rendering. It can also clean up resources. It replaces lifecycle methods in functional components.

Q9. Virtual DOM?

Virtual DOM is a lightweight copy of the real DOM. React updates only changed parts instead of the whole UI. This improves performance significantly. It is a key feature of React.

Q10. Conditional rendering?

Conditional rendering allows displaying UI based on conditions. It uses JavaScript expressions like ternary operators. It helps in dynamic UI updates. It is commonly used for login states.

Q11. Keys in React?

Keys help React identify which items in a list have changed. They improve rendering performance. Keys should be unique and stable. They are important in list rendering.

Q12. Event handling?

React handles events like clicks using functions. It uses camelCase syntax like `onClick`. Event handling allows interaction with UI. It is similar to JavaScript but slightly different syntax.

Q13. Controlled component?

A controlled component is a form element controlled by React state. Input values are managed using state. This allows validation and control. It is widely used in forms.

Q14. Lifting state?

Lifting state means moving state to a common parent component. This allows multiple components to share data. It improves data flow. It is useful in sibling communication.

Q15. API calls?

React communicates with backend using API calls via fetch or axios. Data is usually in JSON format. It is stored in state and rendered in UI. Proper error and loading handling is important.

PostgreSQL (15)

Q1. What is PostgreSQL?

PostgreSQL is an open-source relational database system known for reliability and performance. It supports advanced features like indexing and transactions. It is widely used in backend applications.

Q2. What is table?

A table stores data in rows and columns. Each row is a record and each column is a field. Tables are the basic structure in databases. They organize data efficiently.

Q3. Primary key?

A primary key uniquely identifies each record in a table. It cannot have null or duplicate values. It ensures data integrity. It is essential in relational databases.

Q4. Foreign key?

A foreign key links one table to another. It references the primary key of another table. It helps maintain relationships. It ensures referential integrity.

Q5. What is index?

An index improves the speed of data retrieval. It works like a lookup table. It reduces search time significantly. However, it may slow down inserts.

Q6. What is JOIN?

JOIN is used to combine data from multiple tables. It is based on a related column. It helps retrieve meaningful data. It is widely used in queries.

Q7. Types of JOIN?

Common joins are INNER, LEFT, RIGHT, and FULL JOIN. INNER returns matching rows. LEFT returns all from left table. RIGHT returns all from right. FULL returns all data.

Q8. What is normalization?

Normalization organizes data to reduce redundancy. It divides data into related tables. It improves consistency and reduces duplication. It is important in DB design.

Q9. What is query?

A query is a command used to interact with the database. It is written in SQL. It can retrieve, insert, update, or delete data. Queries are essential for DB operations.

Q10. GROUP BY?

GROUP BY groups rows with similar values. It is used with aggregate functions like COUNT or SUM. It helps summarize data. It is useful in reporting.

Q11. HAVING?

HAVING filters grouped data after GROUP BY. It works with aggregate functions. It is different from WHERE. It is used in complex queries.

Q12. WHERE?

WHERE filters rows before grouping. It is used to apply conditions. It works on individual rows. It improves query precision.

A transaction is a group of operations executed together. It ensures consistency of data. If one fails, all are rolled back. It is important for reliability.

Q14. ACID?

ACID stands for Atomicity, Consistency, Isolation, Durability. These properties ensure reliable transactions. They prevent data corruption. They are core DB concepts.

Q15. LIMIT?

LIMIT restricts the number of rows returned. It is useful for pagination. It improves performance by reducing output. It is commonly used in queries.

DSA (20)

Q1. What is DSA?

DSA stands for Data Structures and Algorithms. It helps in organizing data and solving problems efficiently. It is important for writing optimized code. It is widely used in interviews.

Q2. What is algorithm?

An algorithm is a step-by-step procedure to solve a problem. It should be efficient and correct. Good algorithms reduce time and space complexity. They are the core of programming.

Q3. Time complexity?

Time complexity measures how execution time grows with input size. It helps compare algorithms.

Common examples are $O(n)$, $O(\log n)$. It is important for optimization.

Q4. Space complexity?

Space complexity measures memory usage of an algorithm. It depends on input size. Efficient algorithms use less memory. It is important in large systems.

Q5. Big-O?

Big-O notation represents worst-case complexity. It helps analyze performance. It ignores constants and focuses on growth rate. It is widely used in interviews.

Q6. Array vs Linked List?

Arrays provide fast indexing but fixed size. Linked lists allow dynamic size but slower access. Arrays use contiguous memory. Linked lists use pointers.

Q7. Stack?

Stack follows Last In First Out. Operations include push and pop. It is used in recursion and undo operations. It is a simple data structure.

Q8. Queue?

Queue follows First In First Out. Operations include enqueue and dequeue. It is used in scheduling and BFS. It ensures order of processing.

Q9. Binary search?

Binary search works on sorted arrays. It divides search space into halves. It is very efficient with $O(\log n)$. It is faster than linear search.

Q10. Linear search?

Linear search checks each element one by one. It works on unsorted data. It is simple but less efficient. Its complexity is $O(n)$.

Q11. Sorting?

Sorting arranges data in order. It improves searching and processing. Common algorithms include bubble, merge, quick sort. It is widely used.

Q12. Merge sort?

Merge sort uses divide and conquer approach. It divides array and merges sorted parts. It has $O(n \log n)$ complexity. It is stable and efficient.

Q13. Quick sort?

Quick sort uses pivot-based partitioning. It is fast in practice. Average complexity is $O(n \log n)$. Worst case is $O(n^2)$.

Tree?

A tree is a hierarchical data structure. It has nodes and edges. It is used in file systems and databases. It represents relationships.

Q15. Binary tree?

A binary tree has at most two children per node. It is used in searching and sorting. It is a basic tree structure.

Q16. BFS? BFS

explores nodes level by level. It uses a queue. It is used in shortest path problems. It ensures minimum steps.

Q17. DFS?

DFS explores depth first. It uses stack or recursion. It is used in traversal problems. It is memory efficient.

Q18. Hash table?

Hash table stores key-value pairs. It provides fast lookup. It uses hashing function. Average complexity is $O(1)$.

Q19. Collision?

Collision occurs when two keys map to same index. It is handled using chaining or probing. It affects performance. Good hashing reduces collisions.

Q20. Dynamic Programming?

DP solves problems by storing results of subproblems. It avoids recomputation. It improves efficiency. It is used in optimization problems.